Abagbe 1

Aufgabe 1

c. Webstorm Version 2026.2.3
d. Node Version v24.22.0
e.
npm list -g --depth=0 [Prüfung globalität]

npm list: Listet installierte Pakete auf.
-g: Steht für global (prüft also, ob global etwas installiert ist).
--depth=0: Sorgt dafür, dass nur die Hauptpakete angezeigt werden und nicht deren Unterabhängigkeiten, damit die Liste übersichtlich bleibt.

Es sind keine zusätzlichen globalen NPM-Module installiert. Die aufgeführten Pakete (corepack und npm) sind fester Bestandteil der Node.js-Standardinstallation.

ls -la ~ [Heimverzeichnis]
kein npm module ordner im verzeichnis, nur standard Npm

Aufgabe 2 Beantworten Sie die folgenden Fragen zum Demoprojekt ”Usermanager (client only)“

(a) Wo (Stelle im Code) werden die Nutzer abgespeichert?
(a: der Nutzer wird in script.ts zeile 35 gespeichert „this.users.push(newUser);“
(b) Welche Informationen werden pro Nutzer gespeichert?
(b: pro nutzer wird die Id, der vorname, der nachname und das creationdate gespeichert (datum und Uhrzeit
(c) Wozu wird DOM-Manipulation in diesem Projekt eingesetzt?
(c: Die Dom-Manipulation wird zum Speichern des users, zum Löschen des users und zum Ändern der Userdaten eingesetzt. Zusätzlich auch nochzum einblenden der Statusmitteilungen.
(d) Wie werden beim Laden der Anwendung 3 Nutzer angelegt?
(d: Beim Laden der Anwendung werden die 3 Nutzer über den constructor angelegt. Dieser beinhaltet die ersten 3 Beispiel User
(e) Wie werden die Nachrichtenfelder beim Bearbeiten der Nutzerliste dargestellt?
(e: die Nachrichtenfelder werden dargestellt durch die function addMessage
(f) Wozu dient die private Eigenschaft idCounter in der Klasse User?
(f: die private Eigenschaft idCounter der Klasse User dient der fortlaufenden erstellung der Id nummern für jeden User der angelegt wird.
Wo wird idCounter verwendet?
(f: Der idCounter wird im constructor der Klasse User verwendet.
Bei jedem erstellen eines neuen Nutzers wird der Wert der ID zugewiesen und danach um 1 erhöht
Was bedeutet private?
(f: Private ist ein Zugriffsmodifikator, der modifikator bestimmt das auf diese Eigenschaft nur aus dem inneren der Class zugegriffen werden kann.
Wieso muss die Eigenschaft static sein? Welche folge hätte es, wenn die Eigenschaft nicht static wäre?
(f: Die Eigenschaft static muss sein, da dadurch die Id nicht als teil eines einzelnen Users angenommen wird, sondern auf die gesamte Class bezogen ist. Dadurch startet die ID nicht bei jedem User neu bei 1 sondern jeder User bekommt seine zahl aber eine die Klassenbezogen kontinuierlich wächst.
(g) Wieso setzt sich die Nutzerliste beim Neuladen der Seite zurück?
(g: Da die Seite die Daten mit der sie ausgefüllt wurde, nicht mit einem Speicherort verbunden sind, somit startet das Skript von neu, sobald die Seite neu geladen wird
(h) Erklären sie die Verwendung der for-Schleife und der if-Bedingung in der Methode deleteUser aus der Klasse UserList.
(h: Die for-schleife durchläuft das Array und startet bei 0, es erhöht sich so lange um 1, bis die Id des Users der gelöscht werden soll, erreicht wird. Die If bedingung prüft ob die id des Users aus dem Array mit der Indexzahl dieselbe ist wie die id die gelöscht werden soll

Dann wird auf das array zugegriffen, das splice entfernt das Objekt aus dem Array mit der Index Nummer. (i=index nummer, 1= wieviele Elemente ab dieser position gelöscht werden sollen)
Das return bricht die schleife ab, da die id des Users eindeutig ist und aus dem array gelöscht wurde

(i) Per Code wird geprüft, ob Vorname oder Nachname leer sind. Erklären Sie, warum es dennoch möglich ist einen Vornamen einzugeben, welcher ausschließlich aus Leerzeichen besteht.

(i: Ja es ist möglich einen vornamen einzugeben der nur aus leerzeichen besteht, da im code die länge der eingabe geprüft wird, sodass dort nicht nur nichts stehen kann sondern auch ein leerzeichen (länge 1)

```
if (givenName.length == 0 || familyName.length == 0) {
  addMessage("The given name or family name is empty.");
```

(j) Setzen Sie einen Breakpoint an der Stelle, an der ein neuer Nutzer erzeugt wird. Lassen Sie sich seine zugewiesene ID im Debugger anzeigen.

(j:

1. Brakpoint Zeile 34 script.ts
2. Oben neben Debugger Bug rechts ein Javascript Debug erstellen mit der HTML der Seite
3. Debugger starten
4. Seite öffnet sich
5. Namen eingaben
6. Seite stoppt bevor neuer user angelegt wird
7. Im Debug unter Block -> Functions -> User -> idCounter = 4

Aufgabe 3
a: ✓

b: Die Methode trim() von String-Werten entfernt Leerraum an beiden Enden dieses Strings und gibt einen neuen String zurück, ohne den ursprünglichen String zu verändern.

Um einen neuen String zurückzugeben, bei dem Leerraum nur an einem Ende entfernt wurde, verwenden Sie trimStart() oder trimEnd().

1. .trim() in zeile 36 [unter const]
   Datensäuberung
   Die saubere Speicherung. Hier zusändig für das abkappen versehentlich geschriebener Leerzeichen vor oder nach dem Text. So landen sie fehlerfrei in der Liste/Tabelle.
2. Zeile 164 + .trim() [If-funktion beim hinzufügen eines users]
   Die Validierung
   verhindert hier, dass absenden des Formulars mit reinen leerzeichen.
   .trim() kürzt „ „ zu „“, somit nicht größer 0.

c: Zeile 178

```
  document.getElementById("edit_user_form")!.addEventListener("submit", (event: SubmitEvent): void => {
    event.preventDefault();
    let indexEl: HTMLInputElement = document.getElementById("edit user id") as HTMLInputElement;
    let givenNameEl: HTMLInputElement = document.getElementById("edit_user_given_name") as HTMLInputElement;
    let familyNameEl: HTMLInputElement = document.getElementById("edit_user_family_name") as HTMLInputElement;
    let nickNameEl: HTMLInputElement = document.getElementById("edit user nick name") as HTMLInputElement;

    //Werte aus den Input-Feldern speichern
    let givenName: string = givenNameEl.value;
    let familyName: string = familyNameEl.value;
    let nickName: string = nickNameEl.value;
    //Die Kontrolle ob die Felder leer sind, wird hier durchgeführt.
    //Wenn eines der Felder leer ist, wird eine Nachricht hinzugefügt und die Funktion beendet.
    if (givenName.trim().length == 0 || familyName.trim().length == 0 || nickName.trim().length == 0) {
      addMessage("Update canceled. All fields must be filled.");
      return;
    }
    //Das eigentliche Update des Users wird hier durchgeführt.
    let userId: number = parseInt(indexEl.value);
    userList.updateUser(userId, givenNameEl.value, familyNameEl.value, nickNameEl.value);
    renderList();
  });
});
```

Aufgabe 4

Fertig